Team TROOS

Coding standards

ConsultIQ Coding Standards and Team Conventions

Purpose

This document defines the coding standards, architectural conventions, and development practices for ConsultIQ. Its purpose is to ensure consistency, maintainability, readability, and collaboration across the frontend and backend codebases.

The system consists of the following:

- **Frontend:** React + Vite
- **Backend:** NestJS
- **Styling:** Tailwind CSS
- **Database:** PostgreSQL

General Principles

All team members must follow these principles:

- Write clean, readable, and maintainable code
- Keep components and services focused on a single responsibility
- Reuse shared logic and UI components where possible
- Use meaningful naming throughout the codebase

Naming Conventions

Files and Folders

Use **kebab-case** for folders and non-component files.

Examples:

```
consultant-profile/
project-specification/
cv-parsing/
```

Variable & Function

Use **camelCase.**

Examples:

```
createConsultant()
projectBudget
costToCompany
```

Classes & Interfaces

Use **PascalCase**

Examples:

```
ConsultantService
ProjectService
CreateConsultantDto
```

Constants

Use **UPPER_SNAKE_CASE**

Examples:

MAX_UPLOAD_SIZE

DEFAULT_MATCH_WEIGHT

Database tables & columns

Use **snake_case** and plural table names

Examples:

```
consultants
projects
consultant_skills
Project_required_skills
```

API Endpoints

Use lowercase paths and plural REST-style endpoints.

```
GET /consultants
```

```
POST /consultants
GET /projects
POST /projects
```

Backend Standards

Backend Architecture

The backend follows a feature/module-based architecture.

Each feature must contain:

- controller
- service
- DTOs
- entities/models

Backend Folder Structure

```
backend/src/
│
├── controllers/
│   ├── consultant.controller.ts
│   ├── project.controller.ts
│   ├── scoring-config.controller.ts
│   └── cv-parsing.controller.ts
│
├── consultants/
│   ├── services/
│   │   └── consultant.service.ts
│   │
│   │
│   ├── entities/
│   │   ├── consultant.entity.ts
│   │   └── consultant-skill.entity.ts
│   │
│   └── dto/
│       ├── create-consultant.dto.ts
│       └── update-consultant.dto.ts
│
├── projects/
│   ├── services/
│   │   └── project.service.ts
│   │
│   │
```

```
│   ├── entities/
│   │   ├── project.entity.ts
│   │   └── project-skill-requirement.entity.ts
│   │
│   └── dto/
│       ├── create-project.dto.ts
│       └── update-project.dto.ts
│
├── scoring/
│   ├── services/
│   │   ├── scoring-config.service.ts
│   │   └── scoring-engine.service.ts
│   │
│   │
│   ├── entities/
│   │   └── scoring-config.entity.ts
│   │
│   └── dto/
│       └── update-scoring-config.dto.ts
│
├── cv-parsing/
│   ├── services/
│   │   └── cv-parsing.service.ts
│   │
│   └── dto/
│       └── cv-extract.dto.ts
│
├── common/
│   ├── guards/
│   ├── interceptors/
│   ├── decorators/
│   └── types/
│
├── app.module.ts
└── main.ts
```

Architectural Conventions

Controllers

Controllers are responsible only for:

- handling HTTP requests
- validating incoming data
- invoking services
- returning responses

Controllers must not contain business logic.

Services

Services contain:

- business logic
- validation rules
- scoring calculations

DTOs

DTOs (Data Transfer Objects) are used for:

- request validation
- type safety
- controlling API payload structure

Validation must use class-validator.

Entities

Entities represent database models and relationships.

Common Module Usage

The `common/` directory stores shared cross-cutting functionality.

Guards: Authentication and authorisation logic.

Interceptors: Logging, transformation, and response handling.

Decorators: Custom reusable decorators.

Types: Shared interfaces and reusable types.

Naming Conventions

Element	Convention	Example
Files	kebab-case	`consultant.service.ts`
Classes	PascalCase	`ConsultantService`
Variables	camelCase	`projectBudget`

DTOs	PascalCase + suffix	`CreateProjectDto`
Entities	PascalCase + suffix	`ConsultantEntity`

Error handling and logging

- Use proper HTTP status codes
- Throw NestJS exceptions where appropriate
- Use NestJS Logger service

Frontend Standards

Frontend Architecture

The frontend follows a feature-based architecture where each major system domain (consultants, projects, scoring, CV parsing) contains its own pages, components, services, hooks, and types. Shared UI elements and layouts are extracted into reusable component directories to maintain consistency and reduce duplication across the system.

Frontend Folder Structure

```
frontend/src/
│
├── assets/
│   ├── images/
│   └── logos/
│
├── components/
│   │
│   ├── ui/
│   │   ├── button/
│   │   ├── card/
│   │   ├── modal/
│   │   ├── input/
│   │   ├── dropdown/
│   │   ├── badge/
│   │   └── loader/
│   │
│   ├── layout/
│   │   ├── sidebar/
│   │   └── protected-route/  - for role guards and stuff
```

```
│   │
│   └── shared/
│       ├── search-bar/
│       └── confirmation-dialog/
│
├── features/
│   │
│   ├── consultants/
│   │   ├── pages/
│   │   ├── services/
│   │   ├── types/
│   │
│   ├── projects/
│   │   ├── pages/
│   │   ├── services/
│   │   ├── types/
│   │
│   ├── scoring/
│   │   ├── pages/
│   │   ├── services/
│   │   └── types/
│   │
│   ├── cv-parsing/
│   │   ├── pages/
│   │   ├── services/
│   │   └── types/
│   │
│   └── authentication/
│       ├── pages/
│       ├── services/
│       └── types/
├── hooks/
├── lib/
├── routes/
│   └── app-routes.tsx
│
├── styles/
│   ├── globals.css
│   └── theme.css
│
├── App.tsx
├── main.tsx
└── vite-env.d.ts
```

Notes:

globals.css is for:

- Tailwind imports

- CSS reset/base styles
- global body styling
- typography defaults

theme.css is for:

- CSS variables
- brand colours
- reusable theme tokens

Separation of concerns

Folder	Responsibility
pages	screen-level components
components	reusable feature UI
services	API calls
hooks	Reusable logic
types	TypeScript interfaces

Naming Conventions

Element	Convention	Example
Components	PascalCase	`ConsultantCard.tsx`
Hooks	camelCase with `use`	`useConsultants.ts`
Services	kebab-case	`consultant.service.ts`
Pages	kebab-case	`consultant-list-page.tsx`
Types	kebab-case	`consultant.types.ts`

Git and Version Control Standards

Feature branches

```
feature/frontend/consultant-profile
feature/backend/project-specification
feature/integration/cv-parsing
```

Bug fixes

```
fix/validation-error
fix/sidebar-layout
```

Commit Messages

Use clear, consistent commit messages.

```
feat: add consultant creation endpoint
fix: correct project form validation
refactor: simplify scoring service
```

Pull Requests

Every pull request must:

- Have a clear title
- Include a short description
- Be reviewed before merging